

Dynamic Arrays

This worksheet focuses on implementing and using Dynamic Arrays as Lists. If there are library methods you might find useful to carry out an exercise, they'll be listed, along with a link to the API for that method. Note that there are library methods that do a lot of the manipulation we're looking at, but unless you've been told to use them, consider them off-limits!

Building a Basic Dynamic Array

Making a data structure is broken into two stages:

- Creating the structure to store the information
- Writing the methods to provide functionality that works on the information

Creating a `DynamicArrayList` structure to store Strings

1. Create a class called **DynamicArrayList**. It should contain the following **instance** variables:
 - a. An int called *count* or *numElements* – this will track how many elements are stored in the list.
 - b. An array of Strings called *data* or *elements* – this will store your list's data.
2. Create two constructors:
 - a. A default constructor that sets the size of the String array to 10.
 - b. A constructor that takes a single parameter, an int representing the initial size for the String array

*Note: Unlike python, you can create multiple different constructors within a single class. This facilitates different parameter lists and use cases (which we achieve in python using optional parameters). There **is no change in syntax** to facilitate multiple constructors, the only requirement is that the parameter list is different in each one.*

Adding Basic Functionality

Note: When growing your array or shifting chunks of data up or down in an array, you are essentially copying a chunk of data from an array and putting it somewhere else. Sometimes it's a different array, like when you grow an array. Other times, it's into the *same* array, such as when you're performing a shift insert or shift delete and moving the data up or down.

Useful method: [arraycopy\(\)](#) from the System class.

Functionality: This method will copy a specific number of elements from an array, starting at a specified position. It will insert the copied elements into a specified array (either the same array or a different one) at a specified position.

Usage: `System.arraycopy(sourceArray, startPosOfData, destinationArray, posToInsertAt, numElementsToCopy);`

Examples:

```
// This will copy all elements from the original array to the new array
System.arraycopy(data, 0, newArray, 0, numElements);
// This will shift a section of elements in the data array up by one position
System.arraycopy(data, position, data, position+1, (numElements-position));
```

The following methods should be created within the `DynamicArrayList` class:

*Reminder: All methods should include **javdocs**, and you should commit your code after each method has been added!*

1. A **method called `size()`** that:
 - a. Takes no parameters
 - b. Returns the number of elements currently stored in your list's data array.
2. A **method called `get()`** that:
 - a. Takes 1 parameter, the position (an int) of the element to be retrieved
 - b. Checks that the position is within the bounds of the data in the array
 - i. If the position is invalid, throws an `IndexOutOfBoundsException` exception
 - ii. If the position is valid, returns the data in this position
3. A **method called `indexOf()`** that:
 - a. Takes 1 parameter, the element (a String) to find within the `DynamicArrayList`
 - b. Performs a linear search to find a match for the supplied element:
 - i. Loops through the data in the array from the start up as far as the last slot containing data (remember, the array might have blanks at the end – what is the position of the first empty space?) and checks each one to see if it matches the specified element using `.equals()`
 - ii. If it finds a match, it should return the position of the element
 - iii. If it reaches the end of the array and hasn't found a match, it should return -1

4. A method called **ensureCapacity()** that:
 - a. Is a private method
 - b. Takes no parameters
 - c. Checks if there is any space left in the array (i.e. if the length is less than the number of elements).
 - i. If there is no space left, grow the array by:
 1. Creating a new array 10 elements larger than the original
 2. Copying the data from the original array to the new one (either using `arraycopy` or manually copying it)
 3. Replacing the original array with the new array
 - ii. If there is space left, do nothing
5. A method called **add()** that:
 - a. Takes 1 parameter, the String to be stored in your list's data array
 - b. Ensures that there is enough space to fit the new element (use your `ensureCapacity` method to achieve this)
 - c. Adds the new element to the end of the data in the array
 - d. Updates the number of elements currently stored

Using the basics of the `DynamicArrayList`

1. Write a new program (**`ShoppingList.java`**) that:
 - a. Creates an instance of your `DynamicArrayList` class.
 - b. Fills the instance with 10 hard-coded pieces of text using your **`add()`** method
 - c. Loops through the `DynamicArrayList` and displays the content. The loop should be based on your **`size()`** method, and you should retrieve each entry in the list using your **`get()`** method.
 - i. Make sure that ALL of the data that should be in your list appears
 - ii. Make sure that the data appears in the right order!
 - iii. Make sure that you don't end up seeing blanks!
 - d. Prints out the position of the word "Chocolate" (note: it might not be there!)

Reminder: When working with lists of data, we often use for loops as we know how many times we are going to need to iterate!

Providing More Basic Functionality for the `DynamicArrayList`

The following methods should be created within the `DynamicArrayList` class:

1. A method called **`add()`** that:
 - a. Takes 2 parameters:
 - i. The String to be stored in your list's data array
 - ii. The position (an int) in which the data should be added
 - b. Checks that the position is valid
 - i. If the position is invalid, throws an `IndexOutOfBoundsException`
 - c. Checks that there is enough space to fit the new element
 - i. If there is not enough space – grow the array
 - d. Shift-inserts the new element at the specified position
 - e. Updates the number of elements currently stored
2. A method called **`set()`** that:
 - a. Takes 2 parameters:
 - i. The String to be stored in your list's data array
 - ii. The position (an int) the new element should replace
 - b. Checks that the position is valid
 - i. If the position is invalid, throws an `IndexOutOfBoundsException`
 - c. Stores a copy of the element at the specified position in a temporary variable.
 - d. **Overwrites** the element at the specified position with the new one
 - e. Returns the original contents of that position (which you stored in the temporary variable).
3. A method called **`remove()`** that:
 - a. Takes 1 parameter, the position (an int) of the element to be removed
 - b. Checks that the position is valid
 - i. If the position is invalid, throws an `IndexOutOfBoundsException`

REMINDER: It's ok to have more than one method in the same class with the same name, if they have different parameter lists.

- c. Stores the element from the specified position in a temporary variable
- d. Shift-deletes the new element from the specified position
- e. Updates the number of elements currently stored
- f. Returns the deleted element (that you stored in the temporary variable at step c)

Using your new functionality from the *DynamicArrayList*

1. **Amend your *ShoppingList* program** to add the following to the end of your program:
 - a. Use your 2 parameter **add()** method to add “Apples” to your *DynamicArrayList* in position 5.
 - b. Loop through the *DynamicArrayList* and display the content. Confirm that “Apples” appears in position 5, and that the rest of your list is still present.
 - c. Use your **set()** method to set the element in position 3 to be “Pears”.
 - d. Loop through the *DynamicArrayList* and display the content. Confirm that “Pears” appears in position 3, and that the element that WAS in position 3 is no longer in your list.
 - e. Use your **remove()** method to delete the element in position 8 and display it (it should be returned by your **remove()** method)
 - f. Loop through the *DynamicArrayList* and display the content. Confirm that the element that WAS in position 8 is no longer in your list, and that the elements from after that position are still in the list.

Expanding Your Dynamic Array

Now you have the basics of your *DynamicArrayList* (i.e the core features from the List ADT), you can add more features to expand its usefulness!

Providing (Even) More Functionality for the *DynamicArrayList*

The following methods should be created within the *DynamicArrayList* class:

1. A **method called *remove()*** that:
 - a. Takes 1 parameter: the element to be removed (A String)
 - b. Locates the element to be removed in the list.
 - c. If the element can be found:
 - i. Shift-deletes the element from that position (and ONLY that position)
 - ii. Updates the number of elements currently stored
 - iii. Returns true
 - d. If the element cannot be found, returns false.
2. A **method called *contains()*** that:
 - a. Takes 1 parameter: the element to be found (a String)
 - b. Locates the element within the list (bonus points if you can see how to use a method you’ve already written to do this!)
 - i. Returns true if the element can be found
 - ii. Returns false if the element can’t be found
3. A **method called *clear()*** that:
 - a. Takes no parameters
 - b. Deletes all elements in the list
 - i. This can be done with a brute force approach (deleting each element, one at a time) or a more concise one – can you identify the more concise one?
 - c. Resets the number of elements to 0
 - d. Returns nothing
4. A **method called *isEmpty()*** that:
 - a. Takes no parameters
 - b. Returns true if there are no elements in the list
 - c. Returns false if there are 1 or more elements in the list

Remember, it’s ok to have two methods called **remove()** in the same class, because they have different parameters.

Using your new expanded functionality from the *DynamicArrayList*

1. **Amend your *ShoppingList* program** to add the following to the end of your program:
 - a. Use your **isEmpty()** method to display whether or not there is anything in your list.
 - b. Use your **remove()** method to try and delete “Apples” from the list. Display whether the element could be removed.

- c. Loop through the *DynamicArrayList* and display the content. Confirm that all elements are in the correct place.
- d. Use your **contains()** method to confirm that “Pears” is present in the list.
- e. Use your **clear()** method to wipe the content of the list.
- f. Loop through the *DynamicArrayList* and confirm that there is nothing to be displayed.
- g. Use your **isEmpty()** method to display whether or not there is anything in your list.